

So sánh kiến trúc Transformer và Seq2Seq–Attention cho dịch máy Anh–Việt trên ngữ liệu IWSLT’15

Lê Hoàng Quân

Mã sinh viên: 24022433

github.com/quanai06/machine_translate

TÓM TẮT

Báo cáo cài đặt lại hai kiến trúc Neural Machine Translation (NMT) và so sánh chúng trên cùng ngữ liệu song ngữ Anh–Việt IWSLT’15. Trường hợp thứ nhất là Transformer [1], kiến trúc Encoder–Decoder hoàn toàn dựa trên Self-Attention. Trường hợp thứ hai là Seq2Seq dùng Long Short-Term Memory (LSTM) kèm Global Attention [3], tức kiến trúc nền tảng của bộ mã nguồn tham chiếu TF-NMT [13]. Cả hai được cài đặt bằng PyTorch và dùng chung Tokenizer, chung cách chia Batch, chung công thức tính BLEU, nhằm bảo đảm chênh lệch quan sát được phản ánh kiến trúc chứ không phản ánh kỹ thuật huấn luyện. Trên tập kiểm thử tst2013, mô hình Transformer đạt 30,35 BLEU với Beam Search, mô hình Seq2Seq đạt 27,81 BLEU. Cả hai đều vượt các mốc đã công bố trên cùng ngữ liệu, trong đó bản Seq2Seq vượt chính hệ thống mà nó tái hiện 1,71 điểm BLEU. Báo cáo cũng chỉ ra rằng lợi thế tốc độ huấn luyện thường được gán cho Transformer không xuất hiện ở quy mô mô hình và phân cứng của thí nghiệm này.

Từ khóa: Neural Machine Translation, Transformer, Sequence-to-Sequence, Attention, Byte Pair Encoding, BLEU, IWSLT’15, Anh–Việt, Low-Resource.

MỤC LỤC

1	Mô tả hai trường hợp nghiên cứu	2
1.1	Bài toán và ngữ liệu	2
1.2	Trường hợp 1: Transformer	2
1.3	Trường hợp 2: Seq2Seq kèm Attention	3
1.4	Lựa chọn Framework	3
2	Phương pháp	3
2.1	Các thành phần dùng chung	3
2.2	Trường hợp 1: Transformer	4
2.3	Trường hợp 2: Seq2Seq kèm Attention	4
2.4	Quy trình kiểm chứng cài đặt	5
3	Thực nghiệm và kết quả	5
3.1	Cấu hình thí nghiệm	5
3.2	So sánh với các hệ thống trên cùng ngữ liệu	5
3.3	Benchmark chi tiết hai trường hợp	6
3.4	Nhận xét	7
3.5	Biểu đồ	8

4 Kết luận	8
4.1 Các kiến trúc và kỹ thuật đã áp dụng	8
4.2 Kết luận chính	9
4.3 Hướng phát triển	9
A Tái hiện kết quả	10

1. MÔ TẢ HAI TRƯỜNG HỢP NGHIÊN CỨU

1.1. Bài toán và ngữ liệu

Bài toán là dịch tự động từ tiếng Anh sang tiếng Việt ở miền phụ đề hội thoại TED Talks. Ngữ liệu là IWSLT’15 English–Vietnamese, được phát hành kèm hệ thống Stanford-NMT [4]. Đây là ngữ liệu chuẩn cho cặp ngôn ngữ này và được hầu hết các nghiên cứu sau đó dùng làm mốc đối chiếu.

Bảng 1: Phân chia ngữ liệu IWSLT’15 English–Vietnamese

Tập	Tệp	Số câu	Vai trò
Huấn luyện	<code>train.{en,vi}</code>	133.317	Training
Phát triển	<code>tst2012.{en,vi}</code>	1.553	Validation, chọn Checkpoint
Kiểm thử	<code>tst2013.{en,vi}</code>	1.268	Test, báo cáo BLEU

Sau khi loại các câu dài quá 100 đơn vị Subword, tập huấn luyện thực tế còn 132.010 câu. Ngữ liệu được phát hành ở dạng đã tách từ theo quy ước Moses, và toàn bộ điểm BLEU trong tài liệu tham khảo đều báo cáo trên `tst2013`. Báo cáo giữ nguyên quy ước này để bảo đảm khả năng đối chiếu.

Bảng 2: Thống kê ngữ liệu huấn luyện

Phía	Câu	Token	Vocab thô	Độ dài TB	Hapax
Tiếng Anh	133.317	2.706.255	54.169	20,3	21.912 (40%)
Tiếng Việt	133.317	3.311.508	25.615	24,8	11.298 (44%)

Bảng 2 cho thấy hai đặc điểm chi phối các quyết định thiết kế ở Mục 2. Thứ nhất, 40% từ vựng phía tiếng Anh chỉ xuất hiện đúng một lần (Hapax Legomenon). Với từ điển cố định ở mức từ, phần lớn số này sẽ trở thành ký hiệu `<unk>`; đây chính là Rare Word Problem mà BPE [2] được đề xuất để giải quyết. Thứ hai, câu tiếng Việt dài hơn câu tiếng Anh khoảng 22% tính theo đơn vị phân tách bằng khoảng trắng, do tiếng Việt viết rời từng âm tiết.

1.2. Trường hợp 1: Transformer

Transformer [1] là kiến trúc Encoder–Decoder hoàn toàn dựa trên Self-Attention, không dùng thành phần hồi tiếp. Điểm mấu chốt nằm ở độ dài đường đi của tín hiệu: trong mạng hồi tiếp, thông tin giữa hai vị trí i và j phải truyền qua $|i - j|$ bước, trong khi Self-Attention chỉ cần một bước. Hệ quả là Gradient không suy giảm theo khoảng cách, đồng thời toàn bộ chuỗi đích có thể huấn luyện song song thay vì tuần tự.

Nguồn tham chiếu ban đầu là notebook `demo_transformer.ipynb`.

1.3. Trường hợp 2: Seq2Seq kèm Attention

Trường hợp thứ hai là mạng Encoder–Decoder dùng LSTM kèm Global Attention [3], kiến trúc đứng sau bộ mã nguồn TF-NMT [13] vốn công bố mốc 26,1 BLEU cho cặp Anh–Việt trên tst2013.

Kiến trúc này là kết quả của một chuỗi cải tiến. Seq2Seq [14] đề xuất khung Encoder–Decoder trong đó câu nguồn được nén vào một vector kích thước cố định, nhưng chất lượng suy giảm mạnh theo độ dài câu vì một vector không đủ chứa thông tin của câu dài. RNNSearch [10] gỡ nút thắt này bằng cách cho Decoder truy cập lại toàn bộ trạng thái Encoder thông qua trọng số Attention học được. Global Attention [3] sau đó đơn giản hoá hàm tính điểm và bổ sung cơ chế Input Feeding.

Cần lưu ý rằng TF-NMT [13] được viết bằng TensorFlow 1.x và phụ thuộc vào mô-đun `tf.contrib.seq2seq`. Mô-đun `tf.contrib` đã bị loại bỏ khỏi TensorFlow 2.0, còn TensorFlow Addons, nơi các API này chuyển sang, đã kết thúc vòng đời vào tháng 5 năm 2024. Do đó mã nguồn gốc không còn chạy được trên môi trường hiện tại, và việc cài đặt lại là bắt buộc chứ không phải lựa chọn.

1.4. Lựa chọn Framework

Cả hai trường hợp đều cài đặt bằng PyTorch, vì ba lý do. Thứ nhất, như đã nêu, mã nguồn tham chiếu của trường hợp 2 không còn chạy được nên dù chọn TensorFlow thì vẫn phải viết lại từ đầu. Thứ hai, nếu hai trường hợp dùng hai Framework khác nhau thì mọi chênh lệch về thời gian huấn luyện và bộ nhớ sẽ lẫn giữa yếu tố kiến trúc và yếu tố Framework, khiến phép so sánh mất giá trị. Thứ ba, vòng huấn luyện tường minh của PyTorch cho phép đo chính xác thời gian mỗi Epoch, Throughput và bộ nhớ đỉnh, vốn là một phần yêu cầu của bài toán.

2. PHƯƠNG PHÁP

2.1. Các thành phần dùng chung

Điều kiện tiên quyết để phép so sánh có giá trị là hai mô hình được đối xử như nhau ở mọi khâu ngoài kiến trúc. Bảng 3 liệt kê các thành phần dùng chung và căn cứ tương ứng.

Bảng 3: Các thành phần dùng chung cho cả hai trường hợp

Thành phần	Cấu hình	Căn cứ
Tokenizer	SentencePiece BPE, 8.000 đơn vị mỗi phía	[2]
Tách từ tiếng Việt	Không dùng	[5, 6]
Chia Batch	Theo số Token, tối đa 8.192 Token đích	[7]
Hàm mất mát	Cross-Entropy, Label Smoothing $\varepsilon = 0,1$	[1]
Mixed Precision	fp16 qua <code>torch.amp</code>	[7]
Length Penalty	$((5 + Y)/6)^\alpha$, $\alpha = 1,0$	[9]
Chọn Checkpoint	Theo BLEU trên tst2012, Early Stopping sau 5 Epoch	—

Ba lựa chọn cần giải thích thêm.

Kích thước Vocab. Mức 8.000 Subword mỗi phía được chọn theo quy mô dữ liệu chứ không theo mặc định thông dụng. Với 133 nghìn câu, mức 32.000 Subword là quá lớn vì các Subword hiếm sẽ chỉ xuất hiện vài lần trong toàn bộ quá trình huấn luyện.

Bỏ bước tách từ tiếng Việt. Kết quả thực nghiệm trên chính cặp Anh–Việt [5] cho thấy bước tách từ (Word Segmentation) là không cần thiết đối với NMT dùng Subword, dù nó có ích rõ rệt với Statistical Machine Translation (SMT). Kết luận này được kiểm chứng lại trong nghiên cứu mở rộng [6].

Cách tính BLEU. Ngữ liệu phát hành ở dạng đã tách từ theo Moses, và mọi con số công bố trên ngữ liệu này là Tokenized BLEU tính bằng `multi-bleu.perl`. Công cụ `sacrebleu` mặc định tokenize lại bằng 13a, nên

đưa thẳng văn bản đã tách từ vào sẽ cho một con số thứ ba không so được với gì. Báo cáo dùng sacrebleu với tham số tokenize="none", tương đương multi-bleu.perl, và bổ sung thêm Detokenized BLEU cùng chrF2.

2.2. Trường hợp 1: Transformer

Bảng 4: Siêu tham số của mô hình Transformer

Tham số	Giá trị	Căn cứ
d_{model} / số Head	512 / 4	[1]
Số lớp	6 Encoder + 6 Decoder	[1]
d_{ff}	1024	giảm nửa so với Base
Dropout	0,3	cao hơn mức 0,1 của Base
Chuẩn hoá	Pre-Norm	[8]
Positional Encoding	Sin/Cos, không học	[1]
Weight Tying	Có	—
Learning Rate Schedule	Inverse-Sqrt, 4.000 Warmup	[1]
Optimizer	AdamW, $\beta = (0,9; 0,98)$	[1]
Tổng tham số	39.737.344	

Hai điều chỉnh so với cấu hình gốc cần được giải thích.

Không dùng cấu hình Base. Cấu hình Base [1] có $d_{\text{ff}} = 2048$, Dropout 0,1 và khoảng 65 triệu tham số, được huấn luyện trên WMT’14 English–German với 4,5 triệu câu. IWSLT’15 chỉ có 133 nghìn câu, ít hơn 34 lần. Sao chép nguyên cấu hình từ bối cảnh dữ liệu lớn sang bối cảnh Low-Resource dẫn tới Overfitting. Cấu hình dùng ở đây thu hẹp mạng Feed-Forward xuống một nửa và nâng Dropout lên 0,3.

Dùng Pre-Norm thay vì Post-Norm. Đặt Layer Normalization trước mỗi Sublayer giúp huấn luyện hội tụ ổn định ở Learning Rate lớn mà không cần giai đoạn Warmup dài [8]. Điểm đáng chú ý là ở bối cảnh dữ liệu lớn (WMT’14 English–German), Pre-Norm lại làm *giảm* chất lượng. Nghĩa là đây là một đánh đổi phụ thuộc lượng dữ liệu chứ không phải cải tiến phổ quát. Kết quả TwT [8] được đo trực tiếp trên chính ngữ liệu IWSLT’15 English–Vietnamese và lập mốc 32,8 BLEU, cao nhất được công bố ở thời điểm đó.

2.3. Trường hợp 2: Seq2Seq kèm Attention

Cấu hình bám theo mốc chuẩn công bố trong TF-NMT [13].

Bảng 5: Siêu tham số của mô hình Seq2Seq kèm Attention

Tham số	Giá trị	Căn cứ
Encoder	1 lớp Bi-LSTM, 512 đơn vị	[13]
Decoder	2 lớp LSTM, 512 đơn vị	[13]
Attention	Global Attention, hàm điểm general, có Scale	[3, 13]
Input Feeding	Có	[3]
Dropout	0,2	[13]
Gradient Clipping	5,0	[13]
Beam Size	10	[13]
Optimizer	Adam, Learning Rate 10^{-3}	xem ghi chú
Tổng tham số	20.005.889	

Ba thành phần được lấy trực tiếp từ Global Attention [3].

Hàm tính điểm general. Điểm Attention giữa trạng thái Decoder h_t và trạng thái Encoder h_s được tính theo

$$\text{score}(h_t, h_s) = h_t^\top W_a h_s.$$

Ba biến thể dot, general và concat được khảo sát trong [3]; biến thể general cho kết quả tốt nhất và cũng là mặc định của TF-NMT [13]. Bản cài đặt trong báo cáo có đủ cả ba, mặc định dùng general.

Input Feeding. Vector Attentional $\tilde{h}_t$ được nối vào đầu vào của bước sinh kế tiếp, giúp Decoder ghi nhớ vị trí đã căn chỉnh ở bước trước, qua đó tránh dịch lặp hoặc bỏ sót. Đây là điểm khác biệt so với RNNSearch [10], và cũng là nguyên nhân buộc Decoder chạy tuần tự theo từng bước thời gian.

Length Penalty. Log-Probability luôn giảm theo độ dài chuỗi nên Beam Search thuần thiên vị câu ngắn, kéo Brevity Penalty của BLEU xuống. Công thức chuẩn hoá $((5 + |Y|)/6)^\alpha$ của GNMT [9] được áp dụng cho cả hai trường hợp.

Ghi chú về Optimizer. TF-NMT [13] dùng Stochastic Gradient Descent (SGD) với Learning Rate 1,0. Báo cáo dùng Adam làm mặc định vì trường hợp 1 cũng dùng Adam; nếu hai mô hình chạy hai họ thuật toán tối ưu khác nhau thì chênh lệch quan sát được sẽ lẫn giữa yếu tố kiến trúc và yếu tố tối ưu hoá. Mã nguồn vẫn cho phép tái hiện đúng cấu hình gốc bằng tham số `--optimizer sgd --lr 1.0`.

2.4. Quy trình kiểm chứng cài đặt

Trước khi huấn luyện đầy đủ, hai kiểm tra được thực hiện và cả hai đều phát hiện lỗi thật.

Kiểm tra Mixed Precision. Lỗi lệch kiểu dữ liệu giữa fp16 và fp32 không biểu hiện khi chạy trên CPU không bật Mixed Precision, nhưng làm hỏng ngay lần huấn luyện đầu tiên trên GPU. Kiểm tra này phát hiện rằng các phép chiếu khởi tạo trạng thái Decoder của trường hợp 2 chạy dưới Autocast nên sinh ra trạng thái fp16, trong khi đầu vào LSTM bị ép về fp32, mà nn.LSTM yêu cầu hai thứ này cùng kiểu.

Kiểm tra Overfitting có chủ đích. Mô hình được yêu cầu học thuộc 120 câu rồi chấm BLEU trên chính 120 câu đó; một cài đặt đúng phải đạt trên 80 BLEU. Kiểm tra này phát hiện lỗi thứ hai: bản cài đặt đầu tiên chuyển nguyên tham số `init_weight = 0.1` của TF-NMT [13] sang PyTorch, khiến Logit lúc khởi tạo có độ lệch chuẩn chỉ khoảng 0,005. Hàm Softmax khi đó gần như phân bố đều tuyệt đối trên 8.000 lớp, vùng Gradient rất phẳng, và mô hình chậm hơn Transformer khoảng 10 lần chỉ để thoát khỏi vùng này. Sau khi khởi tạo lại theo Fan-In của từng lớp, Perplexity sau 200 bước giảm từ 7,8 xuống 2,08. Lỗi này không gây dừng chương trình mà chỉ làm hội tụ chậm, nên một lần huấn luyện dài sẽ che mất nó.

3. THỰC NGHIỆM VÀ KẾT QUẢ

3.1. Cấu hình thí nghiệm

Toàn bộ thí nghiệm chạy trên Google Colab với GPU NVIDIA Tesla T4 (16 GB). Hai mô hình dùng chung Tokenizer, chung cách chia Batch, chung công thức BLEU và cùng tập kiểm thử tst2013.

3.2. So sánh với các hệ thống trên cùng ngữ liệu

Bảng 6 đối chiếu hai mô hình của báo cáo với các công trình khác trên cặp ngôn ngữ Anh–Việt, tất cả đều đánh giá chiều English→Vietnamese trên tập tst2013 của IWSLT’15. Các hệ thống được nhóm theo họ kiến trúc: SMT, NMT dựa trên mạng hồi tiếp, và NMT dựa trên Transformer.

Bảng 6: Đối chiếu các công trình dịch máy Anh–Việt, chiều English→Vietnamese, tập kiểm thử tst2013 (IWSLT’15). Cột *Dữ liệu* là số câu song ngữ dùng để huấn luyện. Số in đậm là kết quả tốt nhất trong mỗi họ kiến trúc; **Ours** là hai mô hình của báo cáo này. [†] dùng tập kiểm thử tst2015 thay vì tst2013. [‡] huấn luyện trên ngữ liệu riêng lớn hơn nhiều và tính điểm bằng sacreBLEU không phân biệt hoa thường, nên không so trực tiếp được với các dòng còn lại.

Hệ thống	Kiến trúc	Đơn vị	Dữ liệu	dev	test (tst2013)	
				tst2012	Greedy	Beam
Statistical Machine Translation						
PBSMT [11]	Phrase-Based + LM 4-gram	Từ (VnTokenizer)	122k	—	—	23,15 [†]
NMT dựa trên mạng hồi tiếp						
Stanford-NMT [4]	RNN + Attention	Từ	133k	—	—	23,3
TF-NMT [13]	LSTM + Global Attn	Từ (17k/7,7k)	133k	23,8	25,5	26,1
SAWR-Base [15]	Seq2Seq baseline	BPE	133k	—	—	28,29
SAWR [15]	Seq2Seq + Syntax-Aware	BPE	133k	—	—	29,09
Ours-S2S	LSTM + Global Attn	BPE 8k	132k	23,96	26,64	27,81
NMT dựa trên Transformer						
Ours-TF	Transformer + Pre-Norm	BPE 8k	132k	26,42	29,47	30,35
TwT [8]	Transformer + ScaleNorm	BPE	133k	—	—	32,8
Huấn luyện trên ngữ liệu lớn hơn, chỉ để tham khảo						
Phan-Vu [6]	Bi-LSTM 1024, 98M tham số	BPE 32k	886k	34,77	—	37,00 [‡]

Bốn ghi chú khi đọc Bảng 6.

PBSMT [11] là hệ thống dự thi cùng đợt đánh giá IWSLT 2015 đã sinh ra ngữ liệu này, in ở trang 80–83 cùng kỷ yếu với Stanford-NMT [4] (trang 76–79). Đóng góp chính không nằm ở mô hình dịch mà ở việc mở rộng dữ liệu đơn ngữ cho Language Model, cụ thể là thu thập **1 GB** văn bản báo điện tử tiếng Việt; ba lần chạy chỉ khác nhau ở lượng dữ liệu đơn ngữ và BLEU tăng đều từ **22,70** lên **23,15**. Con số này đo trên tập TED.tst2015 (1.046 câu) chứ không phải tst2013, nên chỉ mang tính tham chiếu.

SAWR [15] tích hợp biểu diễn từ nhận biết cú pháp (Syntax-Aware Word Representations) vào Encoder, nâng baseline từ **28,29** lên **29,09** BLEU, tức **+0,80** điểm. Đây là mốc hữu ích vì baseline của công trình này (**28,29**) mạnh hơn TF-NMT [13] khá nhiều dù cùng dùng 133 nghìn câu, cho thấy khoảng cách giữa các bản cài đặt Seq2Seq trên cùng ngữ liệu là đáng kể.

Phan-Vu [6] đạt **37,00** BLEU nhưng huấn luyện trên ngữ liệu tự thu thập **886.224** câu, gấp **6,6** lần IWSLT’15, với mô hình **98** triệu tham số. Công trình này còn tính điểm bằng sacreBLEU không phân biệt hoa thường thay vì Tokenized BLEU. Vì cả hai lý do đó, con số 37,00 không so trực tiếp được với các dòng còn lại và chỉ đưa vào để cho thấy trần hiệu năng khi có thêm dữ liệu.

Vị trí của hai mô hình trong báo cáo. **Ours-S2S** đạt **27,81**, vượt TF-NMT [13] là hệ thống mà nó tái hiện, nhưng còn kém baseline Seq2Seq của SAWR [15]. **Ours-TF** đạt **30,35**, vượt cả SAWR [15] và đứng thứ hai trong nhóm huấn luyện trên 133 nghìn câu, chỉ sau TwT [8].

3.3. Benchmark chi tiết hai trường hợp

Bảng 7 trình bày toàn bộ số đo của hai mô hình trong cùng điều kiện. Chỉ số nào càng cao càng tốt được đánh dấu ↑, càng thấp càng tốt đánh dấu ↓.

Bảng 7: Benchmark chi tiết hai trường hợp trên cùng ngữ liệu, cùng phần cứng Tesla T4. Số in đậm là kết quả tốt hơn.

Nhóm	Chi số	Ours-TF	Ours-S2S
<i>Chất lượng</i>	BLEU Greedy, tst2013 ↑	29,47	26,64
	BLEU Beam, tst2013 ↑	30,35	27,81
	BLEU Detokenized (13a) ↑	30,39	27,85
	chrF2 ↑	48,85	46,55
	BLEU tốt nhất trên tst2012 ↑	26,42	23,96
<i>Kích thước</i>	Tổng tham số ↓	39.737.344	20.005.889
	Tham số ngoài Embedding ↓	31.545.344	11.813.889
	Tham số Embedding	8.192.000	8.192.000
<i>Huấn luyện</i>	Thời gian mỗi Epoch, giây ↓	143,8	151,9
	Throughput, Token/giây ↑	23.600	22.564
	Tổng thời gian, phút ↓	95,5	46,1
	Epoch tốt nhất	30	12
	Bộ nhớ GPU đỉnh, GB ↓	8,86	4,85
<i>Suy luận</i>	Thời gian sinh mỗi câu, ms ↓	52,84	1,40

3.4. Nhận xét

Bản cài đặt lại vượt hệ thống gốc. Ours-S2S đạt **27,81** BLEU so với **26,1** BLEU của TF-NMT [13], hơn **1,71** điểm. Hai khác biệt nhiều khả năng giải thích khoảng cách này: dùng BPE [2] **8.000** đơn vị thay cho từ điển mức từ **17.000** và **7.700**, qua đó giảm mạnh tỉ lệ $\langle \text{unk} \rangle$; và dùng Adam thay cho SGD. Kết quả này xác nhận bản cài đặt trung thành với kiến trúc gốc.

Transformer vượt Seq2Seq 2,54 điểm BLEU nhưng dùng gấp đôi tham số. Nếu chỉ tính phần ngoài Embedding thì tỉ lệ là **31,5** triệu so với **11,8** triệu, tức gần **2,7** lần. Khoảng cách chất lượng do đó không hoàn toàn quy về kiến trúc.

Khoảng cách thực tế còn lớn hơn con số đo được. Ours-S2S đã hội tụ thật sự: BLEU trên tập phát triển đạt đỉnh ở Epoch **12**, sau đó chững lại và Early Stopping kích hoạt ở Epoch **17**. Huấn luyện thêm sẽ không cải thiện. Ngược lại, Ours-TF chưa hội tụ khi bị dừng: Epoch cho kết quả tốt nhất chính là Epoch cuối cùng, BLEU trên tập phát triển vẫn tăng ở những Epoch cuối (từ **26,11** lên **26,42**), và cả Training Perplexity lẫn Validation Perplexity đều còn giảm. Do đó **30,35** chưa phải giới hạn của Transformer, còn **27,81** gần như là giới hạn của Seq2Seq.

Thời gian mỗi Epoch gần như bằng nhau, trái với kỳ vọng thông thường. Chênh lệch chỉ **6%** (**143,8** so với **151,9** giây). Kỳ vọng phổ biến là Transformer phải nhanh hơn hẳn nhờ Teacher Forcing song song toàn chuỗi, trong khi Input Feeding [3] buộc Decoder chạy tuần tự. Kết quả đo cho thấy lập luận đó chỉ đúng một nửa: Ours-S2S chỉ có một nửa số tham số, tức một nửa khối lượng tính toán trên mỗi Token, đủ bù phần thiệt do chạy tuần tự; thêm vào đó Self-Attention tốn chi phí $O(T^2)$ theo độ dài câu. Ở quy mô mô hình này và trên phần cứng T4, hai yếu tố triệt tiêu nhau. Đây là ví dụ cho thấy không nên suy ra tốc độ thực thi từ đặc điểm kiến trúc mà không đo.

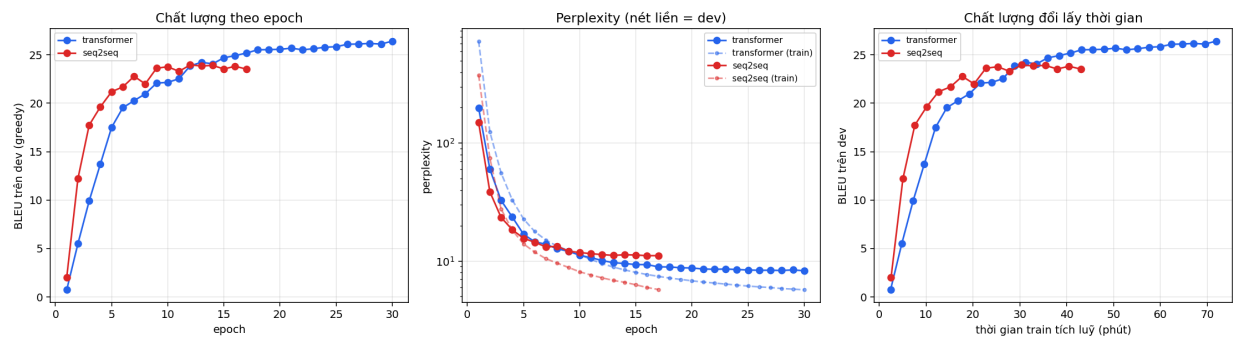
Transformer chậm hơn 37,7 lần khi sinh câu, do hạn chế của bản cài đặt. Cụ thể là **52,84** so với **1,40** mili-giây mỗi câu, tức chậm hơn **37,7** lần. Hàm Decoding trong báo cáo không dùng Key/Value Cache, nên mỗi bước sinh phải chạy lại toàn bộ Decoder trên toàn bộ tiền tố đã sinh, dẫn tới chi phí $O(T^2)$. Mô hình LSTM không gặp vấn đề này vì Hidden State đã tóm tắt sẵn toàn bộ quá khứ trong một vector kích thước cố định. Cần nhấn mạnh đây là hạn chế của cài đặt chứ không phải của kiến trúc Transformer nói chung: các thư viện sản xuất như fairseq [7] đều có Key/Value Cache và không thể hiện khoảng cách này.

Beam Search cải thiện cả hai mô hình. Ours-TF tăng **0,88** điểm khi chuyển từ Greedy sang Beam Size

5; Ours-S2S tăng 1,17 điểm với Beam Size 10. Mức cải thiện nằm trong khoảng thông thường và xác nhận Length Penalty [9] hoạt động đúng.

Chênh lệch dev–test nhất quán với đặc tính ngữ liệu. Ours-TF có 26,42 trên tst2012 và 29,47 trên tst2013, chênh 3,05 điểm; Ours-S2S có 23,96 và 26,64, chênh 2,68 điểm. TF-NMT [13] báo cáo 23,8 và 26,1, chênh 2,3 điểm. Cùng chiều và cùng độ lớn, cho thấy tst2013 vốn dễ hơn tst2012 và các con số đo được là đáng tin cậy.

3.5. Biểu đồ



Hình 1: Từ trái sang phải: BLEU trên tập phát triển theo Epoch; Perplexity theo Epoch (nét liền là Validation, nét đứt là Training); BLEU đối lấy thời gian huấn luyện tích lũy.

4. KẾT LUẬN

Báo cáo đã cài đặt lại hai kiến trúc NMT trên cùng ngữ liệu IWSLT’15 English–Vietnamese và so sánh chúng trong điều kiện kiểm soát. Cả hai bản cài đặt đều vượt mốc tham chiếu tương ứng, xác nhận tính trung thành với kiến trúc gốc.

4.1. Các kiến trúc và kỹ thuật đã áp dụng

Bảng 8 liệt kê chính xác thành phần nào trong mã nguồn được lấy từ công trình nào.

Bảng 8: Ảnh xạ giữa công trình tham khảo và mã nguồn

Nguồn	Tên gọi	Thành phần áp dụng	Vị trí
[1]	Transformer	Kiến trúc Encoder–Decoder, Positional Encoding Sin/Cos, Label Smoothing, Inverse-Sqrt Schedule	translate_transformers/
[2]	BPE	Phân đoạn Subword	common/tokenizer.py
[3]	Global Attention	Ba hàm điểm dot/general/concat, Input Feeding	translate_seq2seq/
[4]	Stanford-NMT	Quy ước chia tập train / tst2012 / tst2013	common/data.py
[5]	—	Bỏ bước tách từ tiếng Việt	scripts/prepare.py
[7]	Scaling-NMT	fp16, chia Batch theo Token, Gradient Accumulation	common/data.py, common/engine.py
[8]	TwT	Pre-Norm Residual	translate_transformers/
[9]	GNMT	Length Penalty cho Beam Search	cả hai tập model.py
[13]	TF-NMT	Toàn bộ siêu tham số của trường hợp 2	translate_seq2seq/config.py

Hai kỹ thuật ScaleNorm và FixNorm của TwT [8] chưa được cài đặt. Các phương pháp xử lý Rare Word trong [12] cũng chưa được áp dụng, do vấn đề này đã được giải quyết một phần bằng BPE.

4.2. Kết luận chính

Thứ nhất, Transformer cho chất lượng dịch cao hơn Seq2Seq-LSTM 2,54 điểm BLEU trong cùng điều kiện (30,35 so với 27,81), và khoảng cách thực tế còn lớn hơn vì Transformer chưa hội tụ khi dừng.

Thứ hai, lợi thế tốc độ huấn luyện thường được gán cho Transformer không xuất hiện ở quy mô mô hình và phần cứng của thí nghiệm này. Số tham số và chi phí Self-Attention bậc hai bù trừ cho lợi thế song song hoá.

Thứ ba, chất lượng của một hệ thống dịch máy phụ thuộc nhiều vào các lựa chọn nằm ngoài kiến trúc. Việc chuyển từ từ điển mức từ sang BPE [2], và từ SGD sang Adam, đủ để đưa một kiến trúc từ năm 2015 vượt qua con số công bố của chính nó thêm 1,71 điểm BLEU.

4.3. Hướng phát triển

Ba hướng có thể nâng kết quả. Thứ nhất, huấn luyện **Ours**-TF tới khi hội tụ thật sự, ước tính đạt 31 đến 32 BLEU dựa trên xu hướng của những Epoch cuối. Thứ hai, bổ sung ScaleNorm và FixNorm [8], vốn là phần còn lại của công trình đã lập mốc 32,8 BLEU trên chính ngữ liệu này. Thứ ba, thêm Key/Value Cache cho hàm Decoding của Transformer, nhằm loại bỏ khoảng cách 37,7 lần về tốc độ suy luận vốn không phải bản chất của kiến trúc.

TÀI LIỆU THAM KHẢO

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 5998–6008. <https://arxiv.org/abs/1706.03762>
- [2] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in *Proc. 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, Berlin, 2016, pp. 1715–1725. <https://aclanthology.org/P16-1162/>
- [3] M.-T. Luong, H. Pham, and C. D. Manning, “Effective Approaches to Attention-based Neural Machine Translation,” in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Lisbon, 2015, pp. 1412–1421. <https://aclanthology.org/D15-1166/>
- [4] M.-T. Luong and C. D. Manning, “Stanford Neural Machine Translation Systems for Spoken Language Domains,” in *Proc. 12th International Workshop on Spoken Language Translation (IWSLT)*, Đà Nẵng, 2015, pp. 76–79. <https://aclanthology.org/2015.iwslt-evaluation.11/>
- [5] H.-H. Phan-Vu, V. T. Tran, V. N. Nguyen, H. V. Dang, and P. T. Do, “Towards State-of-the-art English-Vietnamese Neural Machine Translation,” in *Proc. 8th International Symposium on Information and Communication Technology (SoICT)*, 2017. <https://dl.acm.org/doi/10.1145/3155133.3155205>
- [6] H.-H. Phan-Vu, V. T. Tran, V. N. Nguyen, H. V. Dang, and P. T. Do, “Neural Machine Translation between Vietnamese and English: an Empirical Study,” *Journal of Computer Science and Cybernetics*, vol. 35, no. 2, pp. 147–166, 2019. <https://vjs.ac.vn/index.php/jcc/article/view/13233>
- [7] M. Ott, S. Edunov, D. Grangier, and M. Auli, “Scaling Neural Machine Translation,” in *Proc. Third Conference on Machine Translation (WMT)*, Brussels, 2018, pp. 1–9. <https://aclanthology.org/w18-6301/>
- [8] T. Q. Nguyen and J. Salazar, “Transformers without Tears: Improving the Normalization of Self-Attention,” in *Proc. 16th International Conference on Spoken Language Translation (IWSLT)*, Hong Kong, 2019. <https://aclanthology.org/2019.iwslt-1.17/>
- [9] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi et al., “Google’s Neural Machine Translation System: Bridging the Gap between Human and Machine Translation,” arXiv:1609.08144, 2016. <https://arxiv.org/abs/1609.08144>

- [10] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *Proc. International Conference on Learning Representations (ICLR)*, San Diego, 2015. <https://arxiv.org/abs/1409.0473>
- [11] V. H. Tran, H. T. Vu, T. T. Le, L. N. Pham, and V. V. Nguyen, “The English-Vietnamese Machine Translation System for IWSLT 2015,” in *Proc. 12th International Workshop on Spoken Language Translation: Evaluation Campaign (IWSLT)*, Đà Nẵng, 2015, pp. 80–83. <https://aclanthology.org/2015.iwslt-evaluation.12/>
- [12] T.-V. Nguyen, L.-M. Nguyen, P.-T. Nguyen *et al.*, “Overcoming the Rare Word Problem for Low-Resource Language Pairs in Neural Machine Translation,” in *Proc. 6th Workshop on Asian Translation (WAT@ACL)*, Hong Kong, 2019, pp. 207–214. <https://aclanthology.org/D19-5228/>
- [13] TensorFlow, “Neural Machine Translation (seq2seq) Tutorial,” <https://github.com/tensorflow/nmt>
- [14] I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to Sequence Learning with Neural Networks,” in *Advances in Neural Information Processing Systems 27 (NeurIPS)*, 2014, pp. 3104–3112. <https://arxiv.org/abs/1409.3215>
- [15] M. Zhang, Z. Li, G. Fu, and M. Zhang, “Syntax-Enhanced Neural Machine Translation with Syntax-Aware Word Representations,” in *Proc. 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Minneapolis, 2019, pp. 1151–1161. <https://aclanthology.org/N19-1118/>

A. TÁI HIỆN KẾT QUẢ

Mã nguồn: https://github.com/quanai06/machine_translate

```
git clone https://github.com/quanai06/machine_translate.git
cd machine_translate
pip install -r requirements.txt
bash scripts/download_data.sh      # tải và kiểm tra ngữ liệu
python scripts/prepare.py          # train Tokenizer dùng chung
python scripts/check_amp.py        # kiểm tra Mixed Precision
python scripts/sanity_check.py     # kiểm tra Overfitting có chủ đích
python translate_transformers/train.py
python translate_seq2seq/train.py
python compare.py --plots
```

Trên Google Colab, mở notebooks/colab_full.ipynb và chọn GPU T4.

Toàn bộ nhật ký của lần chạy tạo ra các số liệu trong báo cáo được lưu trong notebooks/translate_colab_run.ipynb. Bảng so sánh và biểu đồ do compare.py sinh ra nằm trong thư mục results/.